COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	September 2, 2026	**DATE SUBMITTED:**	September 2, 2026
NAME:	DE VERA, Sophia D.	**INSTRUCTOR:**	Engr. QUEJADO, Jimlord

SEATWORK 7.2 **ALGORITHM ANALYSIS: BEST-CASE AND WORST-CASE TIME COMPLEXITY**

1. PROCEDURE OUTPUT

1. Finding an Element

```
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }
}
```

- Best-Case Time Complexity: **O(1)**
- Worst-Case Time Complexity: **O(n)**
- Explanation: **The best case that could happen in this program is when the element we're looking for is in the first placement in the array. The worst case that could happen is when the element looking for is at the last of the array, making the algorithm to check all the elements.**

2. Printing All Elements

```
for (int i = 0; i < n; i++)
{
    cout << arr[i] << endl;
}
```

- Best-Case Time Complexity: **O(n)**
- Worst-Case Time Complexity: **O(n)**
- Explanation: **The best case is linear complexity because the occurrence that the target element is in the early placement in the list, the program will stop immediately. But it is also the worst because it searches and runs the code up to the very last or the list.**

3. Nested Loops

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        cout << i << " " << j << endl;
    }
}
```

- Best-Case Time Complexity: **$O(n^2)$**

- Worst-Case Time Complexity: **O(n^2)**
- Explanation: **Both loops check the elements one at a time. Although it is easy to look for the target element one at a time, it is also the worst case since it runs for a long time.**

4. Conditional Processing

```
if (n > 100)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i];
    }
}
else
{
    cout << "Small input";
}
```

- Best-Case Time Complexity: **O(1)**
- Worst-Case Time Complexity: **O(1)**
- Explanation: **The best case is that when the target element appears in the early list, the code stops. But it is also the worst case when the target element is at the last list of the array.**

5. Fixed Inner Loop

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < 10; j++)
    {
        cout << i << j;
    }
}
```

- Best-Case Time Complexity: **O(n^2)**
- Worst-Case Time Complexity: **O(n)**
- Explanation: **Because the fastest way to search for the element is to check each element rather than checking and displaying the output one by one.**

6. Two Consecutive Loops

```
for (int i = 0; i < n; i++)
{
    cout << arr[i];
}

for (int j = 0; j < n; j++)
{
    cout << arr[j] * 2;
}
```

- Best-Case Time Complexity: **O(n)**
- Worst-Case Time Complexity: **O(1)**
- Explanation: **Since there are two for loops, the program iterates each element to be printed out and look for the desired outcome. The worst case is O(1) since it requires the element to bre**

printed out, making it inefficient and takes a long runtime.

7. Dividing by Two

```
int x = n;

while (x > 1)
{
    x = x / 2;
}
```

- Best-Case Time Complexity: **O(log n)**
- Worst-Case Time Complexity: **O(1)**
- Explanation: **Inside the while loop, the program requires it to divide in half, which is more efficient than requiring the code to display the output one at a time.**

8. Nested Loop With Different Limits

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < i; j++)
    {
        cout << i << j;
    }
}
```

- Best-Case Time Complexity: **O(n^2)**
- Worst-Case Time Complexity: **O(n^2)**
- Explanation: **Both loops check the elements one at a time. Although it is easy to look for the target element one at a time, it is also the worst case since it runs for a long time.**

9. Early Termination

```
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        cout << "Found";
        break;
    }
}
```

- Best-Case Time Complexity: **O(1)**
- Worst-Case Time Complexity: **O(n)**
- Explanation: **The best case complexity here is constant complexity since once the element is found the program will stop, rather than checking the whole array list which takes a lot of time.**

10. Two-Level Search

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (arr[i] == target)
        {
            break;
        }
    }
}
```

- Best-Case Time Complexity: **O(1)**
- Worst-Case Time Complexity: **O(n^2)**
- Explanation: **Since there is a break loop, once the target element is found the program runtime is lower compared to making it search the entire element.**

11. Linear Loop With Logarithmic Inner Work

```
for (int i = 0; i < n; i++)
{
    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

- Best-Case Time Complexity: **O(log n)**
- Worst-Case Time Complexity: **O(n log n)**
- Explanation: **To further search the target element, especially on a big array list, dividing it in halves would be efficient. Compared to having it as n log n, the program is directed to increment until the target element is found.**

12. Conditional Nested Loop

```
if (arr[0] == target)
{
    cout << "Found";
}
else
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << arr[i] + arr[j];
        }
    }
}
```

- Best-Case Time Complexity: **O(1)**
- Worst-Case Time Complexity: **O(n^2)**
- Explanation: **The outer loop requires the program to stop once the required/target element is found. Compared to using the two for loops in the inner loop where it has to iterate each element until the very last.**

13. Finding the Maximum

```
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

- Best-Case Time Complexity: **O(1)**
- Worst-Case Time Complexity: **O(n)**
- Explanation:

14. Three Nested Loops

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        for (int k = 0; k < n; k++)
        {
            cout << i + j + k;
        }
    }
}
```

- Best-Case Time Complexity:
- Worst-Case Time Complexity:
- Explanation:

15. Comprehensive Analysis

```
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }

    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

- Best-Case Time Complexity:
- Worst-Case Time Complexity:
- Explanation:

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ***idea generation and explanation of concepts***

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.